
An Algorithm for the Machine Calculation of Complex Fourier Series

Author(s): James W. Cooley and John W. Tukey

Source: *Mathematics of Computation*, Apr., 1965, Vol. 19, No. 90 (Apr., 1965), pp. 297-301

Published by: American Mathematical Society

Stable URL: <https://www.jstor.org/stable/2003354>

REFERENCES

Linked references are available on JSTOR for this article:

https://www.jstor.org/stable/2003354?seq=1&cid=pdf-reference#references_tab_contents

You may need to log in to JSTOR to access the linked references.

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at <https://about.jstor.org/terms>



American Mathematical Society is collaborating with JSTOR to digitize, preserve and extend access to *Mathematics of Computation*

JSTOR

An Algorithm for the Machine Calculation of Complex Fourier Series

By James W. Cooley and John W. Tukey

An efficient method for the calculation of the interactions of a 2^m factorial experiment was introduced by Yates and is widely known by his name. The generalization to 3^m was given by Box et al. [1]. Good [2] generalized these methods and gave elegant algorithms for which one class of applications is the calculation of Fourier series. In their full generality, Good's methods are applicable to certain problems in which one must multiply an N -vector by an $N \times N$ matrix which can be factored into m sparse matrices, where m is proportional to $\log N$. This results in a procedure requiring a number of operations proportional to $N \log N$ rather than N^2 . These methods are applied here to the calculation of complex Fourier series. They are useful in situations where the number of data points is, or can be chosen to be, a highly composite number. The algorithm is here derived and presented in a rather different form. Attention is given to the choice of N . It is also shown how special advantage can be obtained in the use of a binary computer with $N = 2^m$ and how the entire calculation can be performed within the array of N data storage locations used for the given Fourier coefficients.

Consider the problem of calculating the complex Fourier series

$$(1) \quad X(j) = \sum_{k=0}^{N-1} A(k) \cdot W^{jk}, \quad j = 0, 1, \dots, N-1,$$

where the given Fourier coefficients $A(k)$ are complex and W is the principal N th root of unity,

$$(2) \quad W = e^{2\pi i/N}.$$

A straightforward calculation using (1) would require N^2 operations where "operation" means, as it will throughout this note, a complex multiplication followed by a complex addition.

The algorithm described here iterates on the array of given complex Fourier amplitudes and yields the result in less than $2N \log_2 N$ operations without requiring more data storage than is required for the given array A . To derive the algorithm, suppose N is a composite, i.e., $N = r_1 \cdot r_2$. Then let the indices in (1) be expressed

$$(3) \quad \begin{aligned} j &= j_1 r_1 + j_0, & j_0 &= 0, 1, \dots, r_1 - 1, & j_1 &= 0, 1, \dots, r_2 - 1, \\ k &= k_1 r_2 + k_0, & k_0 &= 0, 1, \dots, r_2 - 1, & k_1 &= 0, 1, \dots, r_1 - 1. \end{aligned}$$

Then, one can write

$$(4) \quad X(j_1, j_0) = \sum_{k_0} \sum_{k_1} A(k_1, k_0) \cdot W^{j k_1 r_2} W^{j k_0}.$$

Received August 17, 1964. Research in part at Princeton University under the sponsorship of the Army Research Office (Durham). The authors wish to thank Richard Garwin for his essential role in communication and encouragement.

Since

$$(5) \quad W^{jk_1r_2} = W^{j_0k_1r_2},$$

the inner sum, over k_1 , depends only on j_0 and k_0 and can be defined as a new array,

$$(6) \quad A_1(j_0, k_0) = \sum_{k_1} A(k_1, k_0) \cdot W^{j_0k_1r_2}.$$

The result can then be written

$$(7) \quad X(j_1, j_0) = \sum_{k_0} A_1(j_0, k_0) \cdot W^{(j_1r_1+j_0)k_0}.$$

There are N elements in the array A_1 , each requiring r_1 operations, giving a total of Nr_1 operations to obtain A_1 . Similarly, it takes Nr_2 operations to calculate X from A_1 . Therefore, this two-step algorithm, given by (6) and (7), requires a total of

$$(8) \quad T = N(r_1 + r_2)$$

operations.

It is easy to see how successive applications of the above procedure, starting with its application to (6), give an m -step algorithm requiring

$$(9) \quad T = N(r_1 + r_2 + \cdots + r_m)$$

operations, where

$$(10) \quad N = r_1 \cdot r_2 \cdots r_m.$$

If $r_j = s_j t_j$ with $s_j, t_j > 1$, then $s_j + t_j < r_j$ unless $s_j = t_j = 2$, when $s_j + t_j = r_j$. In general, then, using as many factors as possible provides a minimum to (9), but factors of 2 can be combined in pairs without loss. If we are able to choose N to be highly composite, we may make very real gains. If all r_j are equal to r , then, from (10) we have

$$(11) \quad m = \log_r N$$

and the total number of operations is

$$(12) \quad T(r) = rN \log_r N.$$

If $N = r^m s^n t^p \cdots$, then we find that

$$(13) \quad \begin{aligned} \frac{T}{N} &= m \cdot r + n \cdot s + p \cdot t + \cdots, \\ \log_2 N &= m \cdot \log_2 r + n \cdot \log_2 s + p \cdot \log_2 t + \cdots, \end{aligned}$$

so that

$$\frac{T}{N \log_2 N}$$

is a weighted mean of the quantities

$$\frac{r}{\log_2 r}, \quad \frac{s}{\log_2 s}, \quad \frac{t}{\log_2 t}, \quad \cdots,$$

whose values run as follows

r	$\frac{r}{\log_2 r}$
2	2.00
3	1.88
4	2.00
5	2.15
6	2.31
7	2.49
8	2.67
9	2.82
10	3.01.

The use of $r_j = 3$ is formally most efficient, but the gain is only about 6% over the use of 2 or 4, which have other advantages. If necessary, the use of r_j up to 10 can increase the number of computations by no more than 50%. Accordingly, we can find "highly composite" values of N within a few percent of any given large number.

Whenever possible, the use of $N = r^m$ with $r = 2$ or 4 offers important advantages for computers with binary arithmetic, both in addressing and in multiplication economy.

The algorithm with $r = 2$ is derived by expressing the indices in the form

$$(14) \quad \begin{aligned} j &= j_{m-1} \cdot 2^{m-1} + \cdots + j_1 \cdot 2 + j_0, \\ k &= k_{m-1} \cdot 2^{m-1} + \cdots + k_1 \cdot 2 + k_0, \end{aligned}$$

where j_v and k_v are equal to 0 or 1 and are the contents of the respective bit positions in the binary representation of j and k . All arrays will now be written as functions of the bits of their indices. With this convention (1) is written

$$(15) \quad X(j_{m-1}, \cdots, j_0) = \sum_{k_0} \sum_{k_1} \cdots \sum_{k_{m-1}} A(k_{m-1}, \cdots, k_0) \cdot W^{jk_{m-1} \cdot 2^{m-1} + \cdots + jk_0},$$

where the sums are over $k_v = 0, 1$. Since

$$(16) \quad W^{jk_{m-1} \cdot 2^{m-1}} = W^{j_0 k_{m-1} \cdot 2^{m-1}},$$

the innermost sum of (15), over k_{m-1} , depends only on $j_0, k_{m-2}, \cdots, k_0$ and can be written

$$(17) \quad A_1(j_0, k_{m-2}, \cdots, k_0) = \sum_{k_{m-1}} A(k_{m-1}, \cdots, k_0) \cdot W^{j_0 k_{m-1} \cdot 2^{m-1}}.$$

Proceeding to the next innermost sum, over k_{m-2} , and so on, and using

$$(18) \quad W^{j \cdot k_{m-l} \cdot 2^{m-l}} = W^{(j_{l-1} \cdot 2^{l-1} + \cdots + j_0) k_{m-l} \cdot 2^{m-l}},$$

one obtains successive arrays,

$$(19) \quad \begin{aligned} &A_l(j_0, \cdots, j_{l-1}, k_{m-l-1}, \cdots, k_0) \\ &= \sum_{k_{m-l}} A_{l-1}(j_0, \cdots, j_{l-2}, k_{m-l}, \cdots, k_0) \cdot W^{(j_{l-1} \cdot 2^{l-1} + \cdots + j_0) \cdot k_{m-l} \cdot 2^{m-l}} \end{aligned}$$

for $l = 1, 2, \cdots, m$.

Writing out the sum this appears as

$$\begin{aligned}
 A_l(j_0, \dots, j_{l-1}, k_{m-l-1}, \dots, k_0) \\
 (20) \quad &= A_{l-1}(j_0, \dots, j_{l-2}, 0, k_{m-l-1}, \dots, k_0) \\
 &+ (-1)^{j_{l-1}} i^{j_{l-2}} A_{l-1}(j_0, \dots, j_{l-2}, 1, k_{m-l-1}, \dots, k_0) \\
 &\cdot W^{(j_{l-1} \cdot 2^{l-1} + \dots + j_0) \cdot 2^{m-l}}, \quad j_{l-1} = 0, 1.
 \end{aligned}$$

According to the indexing convention, this is stored in a location whose index is

$$(21) \quad j_0 \cdot 2^{m-1} + \dots + j_{l-1} \cdot 2^{m-l} + k_{m-l-1} \cdot 2^{m-l-1} + \dots + k_0.$$

It can be seen in (20) that only the two storage locations with indices having 0 and 1 in the 2^{m-l} bit position are involved in the computation. Parallel computation is permitted since the operation described by (20) can be carried out with all values of $j_0, \dots, j_{l-2}$, and $k_0, \dots, k_{m-l-1}$ simultaneously. In some applications* it is convenient to use (20) to express A_l in terms of A_{l-2} , giving what is equivalent to an algorithm with $r = 4$.

The last array calculated gives the desired Fourier sums,

$$(22) \quad X(j_{m-1}, \dots, j_0) = A_m(j_0, \dots, j_{m-1})$$

in such an order that the index of an X must have its binary bits put in reverse order to yield its index in the array A_m .

In some applications, where Fourier sums are to be evaluated twice, the above procedure could be programmed so that no bit-inversion is necessary. For example, consider the solution of the difference equation,

$$(23) \quad aX(j+1) + bX(j) + cX(j-1) = F(j).$$

The present method could be first applied to calculate the Fourier amplitudes of $F(j)$ from the formula

$$(24) \quad B(k) = \frac{1}{N} \sum_j F(j) W^{-jk}.$$

The Fourier amplitudes of the solution are, then,

$$(25) \quad A(k) = \frac{B(k)}{aW^k + b + cW^{-k}}.$$

The $B(k)$ and $A(k)$ arrays are in bit-inverted order, but with an obvious modification of (20), $A(k)$ can be used to yield the solution with correct indexing.

A computer program for the IBM 7094 has been written which calculates three-dimensional Fourier sums by the above method. The computing time taken for computing three-dimensional $2^a \times 2^b \times 2^c$ arrays of data points was as follows:

* A multiple-processing circuit using this algorithm was designed by R. E. Miller and S. Winograd of the IBM Watson Research Center. In this case $r = 4$ was found to be most practical.

a	b	c	No. Pts.	Time (minutes)
4	4	3	2^{11}	.02
11	0	0	2^{11}	.02
4	4	4	2^{12}	.04
12	0	0	2^{12}	.07
5	4	4	2^{13}	.10
5	5	3	2^{13}	.12
13	0	0	2^{13}	.13

IBM Watson Research Center
Yorktown Heights, New York

Bell Telephone Laboratories,
Murray Hill, New Jersey

Princeton University
Princeton, New Jersey

1. G. E. P. BOX, L. R. CONNOR, W. R. COUSINS, O. L. DAVIES (Ed.), F. R. HIRNSWORTH & G. P. SILITTO, *The Design and Analysis of Industrial Experiments*, Oliver & Boyd, Edinburgh, 1954.

2. I. J. GOOD, "The interaction algorithm and practical Fourier series," *J. Roy. Statist. Soc. Ser. B.*, v. 20, 1958, p. 361-372; Addendum, v. 22, 1960, p. 372-375. MR 21 #1674; MR 23 #A4231.